

Bản sơ về cách tổ chức dữ liệu hợp lý

He Sen

Trường Trung học Nanshan, Miên Dương, Tứ Xuyên

2007

Nguồn gốc: Bản sơ về cách tổ chức dữ liệu hợp lý

IOI China candidate team papers / 2007 / day 2 / He Sen

Abstract

Tin học là một ngành sâu và đang phát triển rất nhanh. Cùng với sự phát triển đó, quan hệ trong đề bài ngày càng phức tạp, gây nhiều khó khăn cho việc giải bài. Tổ chức dữ liệu hợp lý chính là một phương tiện hiệu quả khi đối mặt với những bài toán như vậy.

Bài viết dùng một vài ví dụ kinh điển để tổ chức dữ liệu hợp lý từ hai góc độ: cấu trúc và thứ tự. Mục tiêu là tối ưu mô hình hoặc nâng cao hiệu quả thuật toán. Các ví dụ gồm quy hoạch động, cấu trúc dữ liệu và đồ thị, nhằm nhấn mạnh vai trò của việc tổ chức dữ liệu trong xây dựng mô hình và tối ưu thuật toán.

Từ khóa. Tổ chức dữ liệu; cấu trúc dữ liệu; quy hoạch động; đồ thị; cây; dãy.

1 Dẫn nhập

Xét một ví dụ đơn giản: cho N số, các số có thể rất lớn, rồi trả lời nhiều truy vấn hỏi một số có xuất hiện trong N số đã cho hay không.

Ta có nhiều cách quen thuộc: bảng băm, trie, sắp xếp trước rồi tìm kiếm nhị phân. Những thuật toán này đều giảm lượng công việc bằng cách tổ chức dữ liệu hợp lý. Bảng băm và trie tổ chức lại *hình thức* dữ liệu; còn sắp xếp trước rồi tìm kiếm nhị phân tổ chức lại *thứ tự* dữ liệu. Hai góc độ “hình thức” và “thứ tự” thường không tách rời nhau trong ứng dụng thực tế, mà cần phối hợp linh hoạt.

Ta đã học nhiều cấu trúc dữ liệu kinh điển; bản thân chúng đều là biểu hiện của việc tổ chức dữ liệu hợp lý. Điều này không chỉ giúp tối ưu hiệu suất chương trình, mà đôi khi còn cung cấp một góc nhìn mới để xây dựng mô hình lời giải. Ví dụ đầu tiên dưới đây minh họa điều đó.

2 Ví dụ 1: Kế hoạch mua sắm của Jinming và các bản mở rộng

2.1 Kế hoạch mua sắm của Jinming

Mô tả bài toán

Cho N món đồ. Mỗi món có một giá trị lợi ích nhỏ hơn 50000 và một giá nhỏ hơn 10000. Một món có thể là **món chính**, được mua trực tiếp, hoặc **phụ kiện**, chỉ được mua nếu món chính của nó đã được mua. Mỗi món chính có nhiều nhất hai phụ kiện, và phụ kiện không có phụ kiện cấp dưới.

Hãy mua một số món sao cho tổng giá không vượt quá M và tổng lợi ích là lớn nhất. Giới hạn: $N < 3200, M < 60$.

Phân tích ngắn

Bài toán gọi nhớ tới ba lô 0-1, nhưng khác ở chỗ phụ kiện không thể mua độc lập.

Tổ chức dữ liệu ban đầu. Quan hệ giữa món chính và phụ kiện là quan hệ cây.

Hình 1: các món chính là gốc của những nhóm nhỏ; có món chính không có phụ kiện, có món có một hoặc hai phụ kiện.

Phương án tổ chức dữ liệu 1

Nếu bỏ qua tính đặc thù của dữ liệu và chỉ nhìn theo cấu trúc cây, ta dễ nghĩ tới cách thêm một “siêu món chính” làm cha của mọi món chính ban đầu.

Hình 2: thêm một gốc giả để biến mọi món chính ban đầu thành con của cùng một siêu gốc.

Trên cây này, có thể thiết kế quy hoạch động:

$$\text{cost}[a] = \text{giá của món ở đỉnh } a, \quad \text{score}[a] = \text{điểm của món ở đỉnh } a.$$

Đặt $f[a][b]$ là tổng điểm lớn nhất trong cây con gốc a , với tổng chi phí không vượt quá b . Nếu $c_1, \dots, c_k$ là các con của a , chuyển trạng thái dạng thô là

$$f[a][b] = \text{Max}\{f[c_1][b_1] + f[c_2][b_2] + \dots + f[c_k][b_k]\} + \text{score}[a],$$

trong đó $\sum b_i = b - \text{cost}[a]$. Cách liệt kê này rõ ràng kém hiệu quả.

Dùng biểu diễn con trái, anh em phải để biến cây thành cây nhị phân. Gọi $\text{left}[a]$ là con trái, $\text{right}[a]$ là anh em phải. Khi chuyển trạng thái, ta chỉ cần liệt kê số tiền cấp cho con trái:

$$f[a][b] = \text{Max}\{\text{Max}_{b_{\text{left}}}\{f[\text{left}[a]][b_{\text{left}}] + f[\text{right}[a]][b - \text{cost}[a] - b_{\text{left}}]\} + \text{score}[a], \\ f[\text{right}[a]][b]\}.$$

Độ phức tạp lý thuyết là $\mathcal{O}(NM^2)$. Với dữ liệu của bài này, độ phức tạp đó vẫn chưa thật lý tưởng.

Phương án tổ chức dữ liệu 2

Quan sát tính đặc thù của bài:

1. phụ kiện không có phụ kiện;
2. mỗi món chính có nhiều nhất hai phụ kiện.

Vì thế, mỗi nhóm gồm một món chính và các phụ kiện của nó có thể được xem như một đối tượng. Với mỗi nhóm, nhiều nhất có năm phương án:

1. không mua gì;
2. chỉ mua món chính;
3. mua món chính và phụ kiện thứ nhất;

4. mua món chính và phụ kiện thứ hai;
5. mua món chính và cả hai phụ kiện.

Khi đó bài toán trở lại dạng ba lô 0-1. Gọi $cost[i][k]$ là chi phí phương án k của nhóm i , và $weight[i][k]$ là lợi ích tương ứng. Đặt $F[i][j]$ là lợi ích lớn nhất khi xét i nhóm đầu và dùng không quá j tiền. Ta có

$$F[i][j] = \max_k (F[i-1][j - cost[i][k]] + weight[i][k]),$$

với $1 \leq k \leq 5$ và $cost[i][k] \leq j$. Số trạng thái là $\mathcal{O}(NM)$, chi phí chuyển là hằng số, nên thuật toán đạt $\mathcal{O}(NM)$.

2.2 Jinming buồn

Mô tả bài toán

Điều kiện được nói lỏng: một món chính có thể có tùy ý nhiều phụ kiện, nhưng phụ kiện vẫn không có phụ kiện cấp dưới. Giới hạn: $N < 60$, $M < 3200$.

Phân tích

Thuật toán 1 vẫn áp dụng được với độ phức tạp $\mathcal{O}(NM^2)$. Nhưng thuật toán 2 không còn phù hợp vì số phương án của một nhóm có thể tăng theo cấp số tổ hợp. Ta cần tổ chức dữ liệu theo cách khác.

Phương án tổ chức dữ liệu 3

Sắp xếp lại thứ tự các món sao cho mỗi phụ kiện đứng ngay sau món chính của nó, và món chính gần nhất bên trái chính là món mà phụ kiện thuộc về. Khi đó điều kiện cần để mua một phụ kiện là món chính gần nhất phía trước nó đã được mua.

Nhờ giới hạn vị trí này, bài toán vốn là bài trên cây được chuyển thành bài trên dãy.

Đặt $cost[i]$, $weight[i]$ lần lượt là giá và lợi ích của món thứ i trong dãy. Gọi

$$F[i][j][k]$$

là lợi ích lớn nhất từ món i đến món n , dùng không quá j tiền, trong đó $k = 1$ nghĩa là món chính ngay trước i đã được mua, còn $k = 0$ nghĩa là chưa mua.

Nếu món i là món chính:

$$F[i][j][k] = \text{Max}\{F[i+1][j - cost[i]][1] + weight[i], F[i+1][j][0]\}.$$

Nếu món i là phụ kiện, thì

$$\begin{cases} F[i][j][1] = \text{Max}\{F[i+1][j - cost[i]][1] + weight[i], F[i+1][j][1]\}, \\ F[i][j][0] = F[i+1][j][0]. \end{cases}$$

Số trạng thái là $\mathcal{O}(NM)$, chuyển trạng thái hằng số, nên thời gian vẫn là $\mathcal{O}(NM)$.

2.3 Jinming rất buồn

Mô tả bài toán

Tiếp tục nói lỏng: phụ kiện có thể có nhiều cấp, tức một phụ kiện cũng có thể có phụ kiện cấp dưới. Quan hệ vừa rộng vừa sâu.

Phân tích

Thuật toán 1 có thể áp dụng trực tiếp vì nó xử lý dữ liệu dưới dạng cây. Nhưng thuật toán 2 và 3 không dùng trực tiếp được. Tuy nhiên chúng gợi ý một hướng quan trọng: bỏ hình thái cây và tổ chức lại thành dãy.

Nhìn lại thuật toán 3, ý chính là: đặt món chính trước phụ kiện, rồi dùng thêm một chiều trạng thái để chuyển bài toán sang dãy. Với cây nhiều cấp, thứ tự tiền thứ tự của cây tự nhiên thỏa điều kiện “cha đứng trước con”.

Nếu đang ở trạng thái mà món cha của một phụ kiện chưa được mua, thì trước khi gắp món chính kế tiếp, mọi phụ kiện liên quan đều không thể mua. Tổng quát hơn:

Khi xét một cây con, nếu không mua gốc của cây con đó, thì mọi đỉnh trong cây con đều không cần xét nữa.

Kết luận này hiển nhiên trên cây, nhưng giá trị của nó nằm ở chỗ ta dùng nó trên thứ tự tiền thứ tự của cây.

Phương án tổ chức dữ liệu 4

Trước hết lấy thứ tự tiền thứ tự của từng cây và lưu vào cùng một dãy *list*. Đồng thời tính $count[i]$, số đỉnh trong cây con gốc i .

Đặt $F[i][j]$ là lợi ích lớn nhất khi xét từ vị trí i đến n trong dãy tiền thứ tự và dùng không quá j tiền. Với đỉnh ở vị trí i , ta có hai lựa chọn:

- mua nó, rồi tiếp tục xét vị trí kế tiếp;
- không mua nó, rồi bỏ qua toàn bộ cây con của nó.

Phương trình là

$$F[i][j] = \text{Max}\{F[i+1][j - \text{cost}[\text{list}[i]]] + \text{weight}[\text{list}[i]], F[i + \text{count}[\text{list}[i]]][j]\}.$$

Thuật toán vẫn là $\mathcal{O}(NM)$ và giải được bản mở rộng một cách gọn gàng.

Tiểu kết

Qua các phiên bản của bài Jinming, ta thấy mô hình đầu tiên nghĩ ra chưa chắc là mô hình tốt nhất. Thuật toán cây ban đầu phải liệt kê cách phân bổ tiền cho các con, đây là nút thắt của quy hoạch động trên cây. Khi tổ chức cây thành dãy tiền thứ tự, phần tiền chưa dùng tự nhiên được chuyển cho các phần tử phía sau, tránh được việc liệt kê phân bổ cho từng nhánh. Đó chính là hiệu quả của tổ chức dữ liệu hợp lý.

3 Ví dụ 2: Quả trên cây

Mô tả bài toán

Cho một cây có gốc gồm N đỉnh, gốc là đỉnh 1. Mỗi đỉnh có một trọng số. Với mỗi đỉnh, cần tính:

1. số đỉnh trong cây con của nó có trọng số lớn hơn nó;
2. số đỉnh không thuộc phần hậu duệ của nó có trọng số lớn hơn nó;
3. số đỉnh trên đường từ gốc tới nó có trọng số lớn hơn nó.

Giới hạn: $1 \leq N \leq 10^5$.

Phân tích

Hai đại lượng sau có thể tính tương đối tự nhiên trong $\mathcal{O}(N \log N)$:

- Với số đỉnh ngoài cây con có trọng số lớn hơn: sắp xếp theo trọng số, rồi lấy số lượng trọng số lớn hơn và trừ đi đáp án của phần trong cây con.
- Với số đỉnh trên đường gốc–đỉnh hiện tại có trọng số lớn hơn: dùng cây đoạn hoặc Fenwick trên trọng số đã rời rạc hóa; DFS trên cây, thêm đỉnh hiện tại khi đi xuống và xóa khi quay lui.

Khó khăn chính là đại lượng thứ nhất: số đỉnh trong cây con có trọng số lớn hơn đỉnh hiện tại. Cách ngây thơ $\mathcal{O}(N^2)$ không chịu nổi $N = 10^5$. Cây đoạn và Fenwick vốn là cấu trúc thống kê trên dãy, nên thay vì sửa cấu trúc dữ liệu cho cây, ta sửa hình thái dữ liệu: chuyển cây thành dãy bằng thứ tự tiền thứ tự.

Trong thứ tự tiền thứ tự, mọi đỉnh của cùng một cây con tạo thành một đoạn liên tiếp. Vì vậy bài toán trở thành:

Cho một dãy, mỗi phần tử có trọng số. Với mỗi phần tử, thống kê trong một đoạn cho trước có bao nhiêu phần tử có trọng số lớn hơn nó.

Phương án tổ chức dữ liệu 1

Dùng trực tiếp dãy tiền thứ tự và ép một cấu trúc dữ liệu lồng nhau để thống kê: cây đoạn mà mỗi nút lưu một danh sách có thứ tự theo trọng số của đoạn tương ứng.

Tiền xử lý có thể làm bằng cách trộn như mergesort, tốn $\mathcal{O}(N \log N)$. Với một truy vấn đoạn độ dài L , ta phân rã đoạn thành không quá $2 \log L$ đoạn con chuẩn. Trên mỗi đoạn con, dùng tìm kiếm nhị phân trong danh sách đã sắp để đếm số phần tử lớn hơn trọng số cần xét. Một truy vấn tốn $\mathcal{O}(\log^2 N)$, toàn bộ tốn $\mathcal{O}(N \log^2 N)$ thời gian và $\mathcal{O}(N \log N)$ bộ nhớ.

Phương án tổ chức dữ liệu 2

Xét trường hợp đặc biệt: nếu trong danh sách hiện tại, mọi phần tử đều có trọng số lớn hơn phần tử k , thì đáp án chỉ là số phần tử trong đoạn cây con của k . Việc đếm số phần tử trong đoạn có thể làm bằng cây đoạn hoặc Fenwick.

Vậy làm sao bảo đảm các phần tử hiện có đều lớn hơn k ? Ta tổ chức lại thứ tự xử lý: sắp mọi đỉnh theo trọng số giảm dần, rồi xử lý lần lượt. Trước khi chèn một đỉnh vào cấu trúc dữ liệu, ta truy vấn đoạn cây con của nó. Do các đỉnh đã chèn đều có trọng số lớn hơn, số phần tử hiện có trong đoạn chính là số đỉnh cần đếm.

Như vậy ta có thuật toán $\mathcal{O}(N \log N)$. Bản chất là tổ chức lại *thứ tự thao tác* trên dữ liệu.

4 Ví dụ 3: Quy hoạch đường bay

Mô tả bài toán

Cho một đồ thị vô hướng có N đỉnh, M cạnh; có thể có nhiều cạnh giữa hai đỉnh. Sau đó có Q lệnh:

- 1 A B: xóa một cạnh giữa A và B ;
- 2 A B: hỏi giữa A và B có bao nhiêu cạnh then chốt, tức xóa cạnh ấy thì A và B không còn liên thông.

Dữ liệu bảo đảm tại mọi thời điểm đồ thị vẫn liên thông. Giới hạn:

$$1 \leq N \leq 3 \cdot 10^4, \quad 1 \leq M, Q \leq 10^5.$$

Phân tích

Cạnh then chốt chính là cầu của đồ thị. Số cầu chỉ ở mức $\mathcal{O}(N)$, nhưng nếu mỗi lần xóa cạnh lại tính lại các thành phần hai cạnh liên thông thì vẫn quá chậm.

Trước hết, co mỗi thành phần hai cạnh liên thông thành một đỉnh. Đồ thị mới là một cây. Khi đó:

- nếu A, B nằm trong cùng một thành phần, giữa chúng không có cạnh then chốt;
- nếu nằm ở hai thành phần khác nhau, đáp án là khoảng cách giữa hai đỉnh tương ứng trên cây.

Với cây, nếu

$$\text{Depth}[x] = \text{độ sâu của } x, \quad \text{LCA}(A, B) = \text{tổ tiên chung gần nhất},$$

thì

$$\text{Dis}(A, B) = \text{Depth}[A] + \text{Depth}[B] - 2 \text{Depth}[\text{LCA}(A, B)].$$

Khi co một thành phần thành một đỉnh, thực chất ta lấy đỉnh có độ sâu nhỏ nhất trong thành phần làm đại diện, tức nâng các đỉnh còn lại lên.

Vấn đề là đồ thị thay đổi do thao tác xóa cạnh. Xóa cạnh có thể làm một khối tách ra, khó xử lý nhanh. Nhưng nếu đảo ngược thời gian, thao tác xóa cạnh trở thành thêm cạnh. Trên cây, thêm một cạnh sẽ tạo chu trình, và mọi đỉnh trên chu trình đó được hợp nhất thành một thành phần hai cạnh liên thông mới.

Giả sử thêm cạnh AB , đặt $T = \text{LCA}(A, B)$. Chu trình gồm đường từ A đến T và đường từ B đến T . Ta cần nâng các đỉnh trên chu trình lên độ sâu $\text{Depth}[T]$, đồng thời nâng cả các cây con đi kèm với chúng.

Để cài đặt:

- LCA có thể xử lý bằng thứ tự Euler kết hợp RMQ; thêm cạnh không làm thay đổi cây cơ sở dùng để tính LCA.
- Thao tác nâng cả một cây con có thể chuyển thành trừ cùng một giá trị trên một đoạn liên tiếp của thứ tự tiền thứ tự; dùng cây đoạn hoặc Fenwick để làm trong $\mathcal{O}(\log N)$.

Đây là ví dụ kết hợp cả hai mặt: co hình thái đồ thị thành cây, rồi đảo thứ tự thời gian để biến xóa cạnh thành thêm cạnh. Trong chi tiết cài đặt, thứ tự Euler và tiền thứ tự lại tiếp tục chuyển các thao tác trên cây thành thao tác trên dãy.

5 Tổng kết

Tổ chức dữ liệu hợp lý xuất hiện ở khắp nơi. Nó không chỉ là một kỹ thuật, mà còn là một hướng suy nghĩ trong thi tin học. Khi quan hệ dữ liệu ngày càng phức tạp và mô hình lời giải không còn rõ ràng, biết tổ chức dữ liệu thường đưa ta rất gần tới lời giải.

Khi được nghe một thuật toán khéo léo, điều đáng quan tâm không chỉ là thuật toán đó, mà còn là cách người thiết kế nghĩ ra nó. Hãy suy nghĩ nhiều, tổng kết nhiều, dám khám phá và không ngừng đổi mới.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang, *Nghệ thuật thuật toán và thi tin học*.

2. He Lin, *Đơn giản hóa quan hệ dữ liệu*, luận văn đội tuyển quốc gia 2005.

Lời cảm ơn

Tác giả cảm ơn thầy Ye Shifu đã hướng dẫn và giúp đỡ; cảm ơn Gu Nan và Wang Xiaoke đã góp ý cho bài viết.

6 Phụ lục: các đề bài được trích dẫn

Kế hoạch mua sắm của Jinming

Jinming được mẹ cho phép mua đồ cho phòng riêng với tổng chi phí không vượt quá N . Mỗi món có giá $v[j]$, độ quan trọng $w[j]$ từ 1 đến 5, và có thể là món chính hoặc phụ kiện. Nếu mua phụ kiện thì phải mua món chính tương ứng. Mỗi món chính có tối đa hai phụ kiện. Cần chọn một danh sách mua sao cho

$$v[j_1]w[j_1] + v[j_2]w[j_2] + \dots + v[j_k]w[j_k]$$

là lớn nhất.

Input gồm N, m , sau đó m dòng v, p, q . Nếu $q = 0$ thì món là món chính; nếu $q > 0$ thì món là phụ kiện của món q . Output là giá trị lớn nhất.

Input mẫu.

```
1000 5
800 2 0
400 5 1
300 5 1
400 3 0
500 2 0
```

Output mẫu.

```
2200
```

Quả trên cây

Một cây ăn quả có $n \leq 10^5$ chỗ phân nhánh, gồm cả gốc. Mỗi chỗ có một quả với độ ngon khác nhau. Với mỗi quả i , nếu cắt bỏ quả đó thì cây được chia thành phần trên, phần dưới và đường từ gốc tới quả đó. Cần tính số quả ngon hơn quả i trong ba vùng tương ứng.

Input cho cha của từng đỉnh và độ ngon của từng quả; output gồm n dòng, mỗi dòng ba số theo thứ tự: phần trên, phần dưới, đường gốc–đỉnh.

Input mẫu.

```
4
1
1
```

1
2
3
4
1
3

Output mẫu.

2 0 0
0 0 0
0 3 1
0 1 1

Quy hoạch đường bay

Trong hệ sao Samuel có N hành tinh và M tuyến bay hai chiều. Một tuyến bay được gọi là then chốt đối với hai hành tinh A, B nếu thiếu tuyến đó thì không thể đi từ A tới B . Các tuyến bay có thể bị phá hủy dần, nhưng dữ liệu bảo đảm đồ thị luôn liên thông. Với mỗi truy vấn, cần xuất số tuyến bay then chốt giữa hai hành tinh.

Input gồm đồ thị ban đầu, sau đó các lệnh: $C = 1$ là hỏi, $C = 0$ là phá hủy tuyến A, B , $C = -1$ là kết thúc. Tổng số lệnh phá hủy và hỏi không vượt quá 40000. Giữa hai hành tinh có nhiều nhất một tuyến trực tiếp.

Input mẫu.

5 5
1 2
1 3
3 4
4 5
4 2
1 1 5
0 4 2
1 5 1
-1

Output mẫu.

1
3